

FLUXO OFICIAL DE GITHUB E PULL REQUEST

Marketplace Comunitário Raízes do Araguaia

1. Objetivo do Fluxo

Este documento define o padrão oficial de uso do GitHub para toda a equipe.

O objetivo é:

- evitar conflitos
- impedir quebra da branch main
- organizar entregas
- padronizar Pull Requests
- melhorar colaboração
- garantir estabilidade do projeto

Todos os membros devem seguir este fluxo obrigatoriamente.

2. Estrutura Oficial de Branches

Branch Principal

```txt id="z4m9jz" main

A branch `main` representa a versão oficial e estável do projeto.

Regras:

- nunca desenvolver diretamente nela
- nunca commitar diretamente nela
- nunca enviar código sem revisão

---

# 3. Branches de Funcionalidade

Cada tarefa deve possuir sua própria branch.

Formato obrigatório:

```
```txt id="b0p6rn"
feat/nome-da-funcionalidade
```

Exemplos:

```
```txt id="g93jhh" feat/guilherme-home feat/flavio-mapa feat/julia-escolinha feat/cecilia-catalogo feat/
marianne-banco feat/henrique-infra
```

```

4. Fluxo Oficial de Desenvolvimento

PASSO 1 – Atualizar Projeto

Antes de iniciar qualquer tarefa:

```bash id="5q5g87"
git checkout main
git pull origin main
```

PASSO 2 — Criar Branch

Criar a branch da tarefa:

```
```bash id="77htn9" git checkout -b feat/nome-da-funcionalidade
```

```

PASSO 3 – Desenvolver

Durante o desenvolvimento:

- alterar apenas sua área
- evitar arquivos globais
- comunicar mudanças estruturais
- fazer commits pequenos
- testar frequentemente

PASSO 4 – Validar Localmente

Antes de qualquer commit final:

```bash id="1v36cq"
```

```
npm run build
npm run lint
```

Também verificar:

- responsividade
- navegação
- console
- erros visuais

PASSO 5 — Commit

Padrão oficial de commits:

```
``txt id="pw9gx6" feat: fix: refactor: docs: style:
```

Exemplos:

```
``bash id="s2o59n"
git commit -m "feat: adiciona cards da home"
git commit -m "fix: corrige filtros do mapa"
git commit -m "refactor: reorganiza hooks do catalogo"
```

PASSO 6 — Push

Enviar branch para GitHub:

```
``bash id="y38f8n" git push origin feat/nome-da-funcionalidade
```

5. Fluxo Oficial de Pull Request

Após finalizar a tarefa:

- abrir Pull Request
- solicitar revisão
- aguardar aprovação
- corrigir ajustes solicitados

Nenhum merge deve acontecer sem revisão.

6. Estrutura Obrigatória do Pull Request

Todo PR deve conter:

O que foi feito
Descrever claramente a funcionalidade.

Arquivos ou módulos alterados
Explicar impacto técnico.

Como testar
Passo a passo para validação.

Dependências
Informar se depende de outro módulo.

Pendências
Informar o que ainda não foi concluído.

7. Modelo Oficial de Pull Request

```md id="1f25bo"

## O que foi feito

Implementado sistema de cards da Home.

## Módulos alterados

- HomePage
- ProductCard
- Categories

## Como testar

1. Rodar npm run dev
2. Abrir Home
3. Validar cards
4. Testar responsividade

## Dependências

Necessita dos dados mockados do catálogo.

## Pendências

Integração final com API.

---

## 8. Arquivos de Alto Risco

Os seguintes arquivos exigem atenção máxima:

```
```txt id="5d6gx4" frontend/src/main.tsx frontend/src/routes/router.tsx frontend/src/index.css frontend/  
package.json frontend/src/shared/
```

Mudanças nesses arquivos devem:

- ser comunicadas
- ser revisadas
- ser testadas cuidadosamente

9. Atualização de Branch

Antes de abrir PR:

```
```bash id="1dgmy0"  
git checkout main
git pull origin main
git checkout sua-branch
git merge main
```

Resolver conflitos localmente antes do PR.

---

## 10. Regras Oficiais de Merge

Um merge só pode acontecer quando:

- build estiver funcionando
  - lint estiver funcionando
  - PR estiver documentado
  - revisão técnica estiver concluída
  - conflitos resolvidos
  - testes básicos concluídos
-

## 11. O Que NÃO Fazer

Proibido:

- desenvolver na main
  - enviar código sem testar
  - alterar área de outro membro sem avisar
  - abrir PR gigante
  - misturar múltiplos módulos
  - ignorar conflitos
  - commitar código quebrado
- 

## 12. Processo de Revisão Técnica

O Tech Lead deve validar:

- padrão de código
  - organização
  - performance
  - integração
  - impacto técnico
  - possíveis conflitos
  - qualidade geral
- 

## 13. Objetivo Final do Fluxo

Esse fluxo existe para garantir:

- estabilidade
- colaboração organizada
- redução de conflitos
- previsibilidade
- qualidade técnica
- escalabilidade
- segurança operacional

Todos os membros devem seguir esse processo durante todo o desenvolvimento do projeto.